

Member Auth API

A .NET 8 Clean Architecture Authentication Service
Code Walkthrough & Design Rationale

.NET 8 (→ .NET 10 ready)

Clean Architecture

CQRS + MediatR

RS256 JWT

EF Core

Refresh-token rotation

Prepared for the Member Portal team

Solution: `c:\ACH\AUTH API`

Contents

1. What this service does & the big picture
2. Clean Architecture — the four layers and why
3. Domain layer
4. Application layer (use cases)
5. Infrastructure layer
6. API layer (the HTTP edge)
7. Security design deep-dive
8. Configuration, mocking & switching to the real DB
9. Project-setup best practices
10. How to run & test
11. Glossary

1. What this service does & the big picture

The Member Portal needs a single, trustworthy place to verify a member's username and password and hand back a token that *other* microservices can trust. This API is exactly that — an **authentication authority**.

The happy path, in plain English

1. Member POSTs username + password to `/api/v1/auth/login`
2. We look the member up in the database and verify the password against a stored salted hash
3. If valid, we mint a short-lived **JWT access token** signed with our private RSA key
 - the token carries the member's identity **plus their LOBs and Plan IDs**
4. We also issue a **refresh token** (so the client can get new access tokens without re-login)
5. Other microservices validate the JWT using only our **public key** (served at the JWKS endpoint)

Why a dedicated Auth API? Centralising authentication means every other service trusts one issuer, password handling lives in exactly one place, and the security rules (lockout, token lifetimes, signing keys) are configured and audited once rather than copied across services.

What's in scope right now

- Login (returns access + refresh tokens with LOB / Plan claims)
- Refresh (rotates the refresh token)
- A protected test endpoint (`/api/v1/members/me`) to prove the token works
- JWKS + discovery endpoints so downstream services can validate tokens

2. Clean Architecture — the four layers and why

The solution is split into four projects. The golden rule is the **Dependency Rule**: source code dependencies only ever point *inwards*, toward the domain. Outer layers know about inner layers, never the reverse.

Api → Infrastructure → Application → Domain

Project	Responsibility	Depends on
AuthApi.Domain	Business entities (Member, Lob, Plan, RefreshToken) and the Result / Error primitives. Pure C#, zero framework references.	nothing
AuthApi.Application	The use cases (Login, Refresh) as CQRS commands, plus the <i>interfaces</i> it needs (database, token service, clock). Defines <i>what</i> happens.	Domain
AuthApi.Infrastructure	The concrete <i>how</i> : EF Core database, RSA token signing, password hashing. Implements the Application's interfaces.	Application
AuthApi.Api	The HTTP edge: endpoints, JWT validation, middleware, Swagger. Wires everything together at startup.	Application + Infrastructure

Why bother splitting it up? Two concrete payoffs for this project:

- **The database is swappable.** The use cases talk to an interface (`IApplicationDbContext`), so we run on a mocked in-memory database today and switch to the real SQL Server later by changing one registration — no business logic changes.
- **The rules are testable.** Login logic has no dependency on ASP.NET or SQL Server, so it can be unit-tested with fakes in milliseconds.

3. Domain layer

The innermost layer. It holds the nouns of the business and a couple of building blocks for expressing success/failure. It references no NuGet packages at all.

3.1 The Result & Error pattern

Instead of throwing exceptions when a login fails, we *return* a value that says "this failed, and here's why". This is the **Result pattern** (a Milan Jovanović staple).

src/Domain/Common/Error.cs

```
public sealed record Error(string Code, string Description, ErrorType Type)
{
    public static readonly Error None = new(string.Empty, string.Empty,
    ErrorType.Failure);
    public static Error Unauthorized(string code, string description) ⇒ new(code,
    description, ErrorType.Unauthorized);
    public static Error Forbidden(string code, string description) ⇒ new(code,
    description, ErrorType.Forbidden);
    // ... Validation, NotFound, Conflict, Failure
}
```

src/Domain/Common/Result.cs (essence)

```
public class Result<TValue> : Result
{
    public TValue Value ⇒ IsSuccess ? _value! : throw new InvalidOperationException(/*
    ... */);

    // Implicit conversions let handlers write `return value;` or `return someError;`
    public static implicit operator Result<TValue>(TValue value) ⇒ Success(value);
    public static implicit operator Result<TValue>(Error error) ⇒ Failure<TValue>
    (error);
}
```

Why not just throw exceptions? "Wrong password" is an *expected* outcome, not an exceptional one. Exceptions are expensive and easy to forget to catch. A **Result** makes the failure part of the method signature, so the compiler and the reader both see it. The **ErrorType** enum later maps cleanly onto HTTP status codes (Unauthorized → 401, etc.) without the Domain ever knowing what HTTP is.

3.2 Entities

Each member can belong to many lines of business and many plans, so we model those as many-to-many relationships through join entities.

src/Domain/Members/Member.cs (selected)

```
public class Member
{
    public Guid Id { get; set; }
    public string Username { get; set; }
    public string PasswordHash { get; set; } // salted hash – never plaintext
    public string PasswordSalt { get; set; }
    public bool IsActive { get; set; } = true;
    public int AccessFailedCount { get; set; }
    public DateTime? LockoutEndUtc { get; set; }

    public ICollection<MemberLob> MemberLobs { get; set; }
    public ICollection<MemberPlan> MemberPlans { get; set; }

    public void RegisterFailedLogin(int maxAttempts, TimeSpan lockoutDuration, DateTime
utcNow)
    {
        AccessFailedCount++;
        if (AccessFailedCount ≥ maxAttempts)
        {
            LockoutEndUtc = utcNow.Add(lockoutDuration);
            AccessFailedCount = 0;
        }
    }
    public IReadOnlyCollection<string> LobCodes() ⇒ /* distinct LOB codes */;
    public IReadOnlyCollection<int> PlanIds() ⇒ /* distinct plan ids */;
}
```

Why put lockout logic on the entity? It is a business rule about a member, so it lives with the member ("rich domain model"). The handler stays thin and the rule is unit-testable in isolation. Storing only a salted hash + salt means a database leak never reveals real passwords.

src/Domain/Members/RefreshToken.cs (selected)

```
public class RefreshToken
{
    public string TokenHash { get; set; } // we store the SHA-256 hash, NOT the
token
    public DateTime ExpiresUtc { get; set; }
    public DateTime? RevokedUtc { get; set; }
    public string? ReplacedByTokenHash { get; set; } // links a rotated token to its
successor
    public bool IsActive(DateTime utcNow) ⇒ RevokedUtc is null && utcNow < ExpiresUtc;
}
```

4. Application layer (use cases)

This layer says *what* the system does, using the **CQRS** pattern with **MediatR**: each use case is a `Command` (the request) handled by a single `Handler`. It also declares the interfaces it needs the outside world to fulfil.

4.1 The interfaces it depends on

Interface	Purpose	Implemented in
<code>IApplicationDbContext</code>	Database access (exposes the EF <code>DbContext</code> s)	Infrastructure → <code>AuthDbContext</code>
<code>IPasswordHasher</code>	Verify a password against the stored salted hash	Infrastructure → <code>SaltedHashPasswordHasher</code>
<code>ITokenService</code>	Create the JWT access token & refresh tokens	Infrastructure → <code>TokenService</code>
<code>IDataTimeProvider</code>	Abstracts "now" so time-based logic is testable	Infrastructure → <code>SystemDateTimeProvider</code>

Why declare interfaces here instead of in Infrastructure? This is the **Dependency Inversion Principle**. The use case owns the contract it needs; Infrastructure adapts to it. That is what keeps the dependency arrow pointing inwards and lets us swap implementations (e.g. a fake clock in tests, or SQL Server instead of in-memory).

4.2 The Login use case

```
src/Application/Authentication/Login/LoginCommand.cs
```

```
public sealed record LoginCommand(string Username, string Password, string? IPAddress)
    : IRequest<Result<AuthenticationResult>>;
```

```
src/Application/Authentication/Login/LoginCommandHandler.cs (core logic)
```

```
var member = await db.Members
    .Include(m => m.MemberLobs).ThenInclude(ml => ml.Lob)
    .Include(m => m.MemberPlans)
    .FirstOrDefaultAsync(m => m.Username == request.Username, ct);

// Unknown user: still run a hash check against a dummy value so the response time is the
// same as for a real user - this prevents attackers from discovering valid usernames by
// timing.
if (member is null)
{
```

```

passwordHasher.Verify(request.Password, DummyHash, DummySalt);
return AuthErrors.InvalidCredentials;           // generic 401
}

if (member.IsLockedOut(now)) return AuthErrors.AccountLocked;           // 403

if (!member.IsActive || !passwordHasher.Verify(request.Password, member.PasswordHash,
member.PasswordSalt))
{
    member.RegisterFailedLogin(_lockout.MaxFailedAttempts, _lockout.LockoutDuration,
now);
    await db.SaveChangesAsync(ct);
    return AuthErrors.InvalidCredentials;           // generic 401
}

member.RegisterSuccessfulLogin(now);
var access = tokenService.CreateAccessToken(member, member.LobCodes(),
member.PlanIds());
var refresh = tokenService.CreateRefreshToken();
db.RefreshTokens.Add(new RefreshToken { /* hash, expiry, member id */ });
await db.SaveChangesAsync(ct);
return new AuthenticationResult(/* member, tokens, lobs, planIds */);

```

Security choices visible here:

- **No user enumeration** — unknown user, wrong password, and inactive account all return the *same* generic `InvalidCredentials`.
- **Timing-attack resistance** — the unknown-user branch performs a dummy hash so it takes about the same time as a real verification.
- **Brute-force throttling** — failed attempts are counted and the account locks after N tries.

4.3 The Refresh use case & rotation

Refreshing exchanges a valid refresh token for a brand-new pair, and **rotates** (invalidates) the old one. If a token that was already rotated is presented again, we treat it as theft and revoke the member's whole chain.

src/Application/Authentication/RefreshToken/RefreshTokenCommandHandler.cs (essence)

```

var presentedHash = tokenService.HashRefreshToken(request.RefreshToken);
var existing = await db.RefreshTokens.Include(...).FirstOrDefaultAsync(rt =>
rt.TokenHash == presentedHash, ct);

if (existing is null) return AuthErrors.InvalidRefreshToken;

if (!existing.IsActive(now))           // already revoked/expired =>
possible reuse
{
    await RevokeAllActiveTokensAsync(existing.MemberId, now, ip, ct); // contain the
breach
    return AuthErrors.InvalidRefreshToken;
}

```

```

}

var newRefresh = tokenService.CreateRefreshToken();
existing.Revoke(now, ip, newRefresh.Hash);           // old token points to its
replacement
db.RefreshTokens.Add(new RefreshToken { TokenHash = newRefresh.Hash, /* ... */ });
var access = tokenService.CreateAccessToken(member, member.LobCodes(),
member.PlanIds());

```

4.4 Validation as a pipeline behavior

Every request passes through a MediatR **pipeline behavior** that runs its `FluentValidation` rules before the handler executes.

`src/Application/Common/Behaviors/ValidationBehavior.cs` (essence)

```

var failures = (await Task.WhenAll(_validators.Select(v => v.ValidateAsync(context,
ct))))
    .SelectMany(r => r.Errors).Where(f => f is not null).ToList();
if (failures.Count != 0) throw new ValidationException(failures);
return await next();

```

Why a behavior? Validation is a cross-cutting concern. Putting it in the pipeline means every current and future use case is validated automatically — handlers never repeat "if username empty...". The thrown `ValidationException` is turned into a clean HTTP 400 by the API's global exception handler.

5. Infrastructure layer

The concrete implementations. This is the only layer that knows about EF Core, RSA keys, SHA hashing, and SQL Server.

5.1 The DbContext and the provider switch

src/Infrastructure/DependencyInjection.cs (persistence)

```
if (provider.Equals("SqlServer", StringComparison.OrdinalIgnoreCase))
{
    var cs = configuration.GetConnectionString("AuthDb")
        ?? throw new InvalidOperationException("ConnectionStrings:AuthDb is
required.");
    options.UseSqlServer(cs, sql => sql.EnableRetryOnFailure());
}
else
{
    options.UseInMemoryDatabase("AuthApiMockDb"); // the mock, used today
}
services.AddScoped<IApplicationDbContext>(sp => sp.GetRequiredService<AuthDbContext>());
```

Why this matters for you: "mock now, real database later" is a single configuration value. Because the rest of the app only knows `IApplicationDbContext`, swapping `UseInMemoryDatabase` for `UseSqlServer` requires *zero* code changes in the Domain, Application, or API layers.

5.2 Password verification — configurable, not hard-coded

The existing member database stores passwords as a salted SHA hash. The exact recipe (SHA-256 vs 512, salt-before vs salt-after, base64 vs hex, iteration count) varies between legacy systems, so we made it configuration-driven rather than baked in.

src/Infrastructure/Security/PasswordHashing/SaltedHashPasswordHasher.cs (essence)

```
public bool Verify(string password, string storedHash, string storedSalt)
{
    var expected = Decode(storedHash, _options.HashEncoding);
    var salt      = Decode(storedSalt, _options.SaltEncoding);
    var actual    = ComputeHash(password, salt); // honours
algorithm/placement/iterations
    return CryptographicOperations.FixedTimeEquals(actual, expected); // constant-time
}
```

Why FixedTimeEquals ? A normal byte-by-byte comparison returns faster when the first byte differs. Attackers can measure that to guess a hash one byte at a time. A constant-time compare

always takes the same duration, closing that side-channel.

Action when connecting the real DB: set the `PasswordHashing` section to match exactly how the legacy system hashed passwords. No code change — just configuration.

5.3 Signing tokens with RSA (RS256)

The `RsaSigningKeyProvider` owns the RSA key. In Development it generates and caches a key locally; in Production you supply one from a secret store. It also exposes the **public** key as a JWKS for other services.

src/Infrastructure/Security/Jwt/TokenService.cs (claims)

```
var claims = new List<Claim>
{
    new(JwtRegisteredClaimNames.Sub, member.Id.ToString()),
    new(JwtRegisteredClaimNames.Jti, jti),                // unique token id
    new(JwtRegisteredClaimNames.UniqueName, member.Username),
};
claims.AddRange(lobCodes.Select(code => new Claim("lob", code))); // one per LOB
claims.AddRange(planIds.Select(id => new Claim("plan", id.ToString()))); // one per plan

var descriptor = new SecurityTokenDescriptor
{
    Subject = new ClaimsIdentity(claims),
    Issuer = _options.Issuer, Audience = _options.Audience,
    Expires = now.Add(_options.AccessTokenLifetime),        // short-lived (15 min default)
    SigningCredentials = keyProvider.SigningCredentials,    // RSA + SHA-256
    TokenType = "at+jwt"
};
```

Why asymmetric RS256 instead of a shared secret (HS256)? With a shared secret, every service that *validates* a token also holds the key that can *mint* tokens — one leak anywhere forges identities everywhere. With RS256, only this Auth API has the private (signing) key; everyone else gets the public key and can only verify. That is the industry-standard model used by Auth0, IdentityServer, Azure AD, etc.

5.4 Refresh tokens are random & stored hashed

src/Infrastructure/Security/Jwt/TokenService.cs (refresh)

```
var raw = Base64UrlEncoder.Encode(RandomNumberGenerator.GetBytes(32)); // 256 bits of entropy
var hash = HashRefreshToken(raw);                                     // SHA-256
// the RAW value is returned to the client once; only the HASH is saved in the DB
```

Why store only the hash? Same principle as passwords — if the database leaks, the stored hashes can't be replayed as tokens. The raw value exists only in transit and in the client.

5.5 Seeding the mock data

`AuthDbDataSeeder` fills the in-memory database with two demo members, three LOBs and three plans on startup (Development only), so the API is testable end-to-end with no real database. It hashes the demo passwords through the very same `IPasswordHasher`, so login behaves identically to production.

6. API layer (the HTTP edge)

Thin by design. It maps HTTP requests onto MediatR commands, configures the security middleware, and composes the whole application at startup. Endpoints are **Minimal APIs** grouped by feature.

6.1 The request pipeline (Program.cs)

src/Api/Program.cs (order matters)

```
app.UseExceptionHandler();           // turn exceptions into clean ProblemDetails
app.UseSecurityHeaders();             // nosniff, DENY framing, no-referrer, ...
if (!app.Environment.IsDevelopment()) app.UseHsts();
app.UseHttpsRedirection();
app.UseSerilogRequestLogging();
app.UseCors(CorsPolicyName);
app.UseRateLimiter();                 // throttle the auth endpoints
app.UseAuthentication();              // who are you? (validate the JWT)
app.UseAuthorization();               // are you allowed?
app.MapAuthEndpoints();
app.MapMemberEndpoints();
app.MapDiscoveryEndpoints();          // /.well-known/jwks.json + openid-configuration
```

Why the order is deliberate: middleware runs top-to-bottom. Exception handling must wrap everything, security headers should apply to every response, and authentication must run before authorization. Getting this order wrong is a classic source of security holes.

Before serving traffic, startup also forces the signing key to be created and (in Dev) seeds data — a **fail-fast** check so a misconfigured key never reaches a live request:

```
using (var scope = app.Services.CreateScope())
{
    scope.ServiceProvider.GetRequiredService<RsaSigningKeyProvider>(); // validates
    Jwt options too
    if (seedMockData) await scope.ServiceProvider.GetRequiredService<AuthDbDataSeeder>
    ().SeedAsync();
}
```

6.2 Endpoints

src/Api/Endpoints/AuthEndpoints.cs (essence)

```
var group = app.MapGroup("/api/v1/auth").RequireRateLimiting(RateLimiterPolicies.Auth);
group.MapPost("/login", LoginAsync).AllowAnonymous();
group.MapPost("/refresh", RefreshAsync).AllowAnonymous();

static async Task<IResult> LoginAsync(LoginRequest req, ISender sender, HttpContext ctx,
```

```

CancellationToken ct)
{
    var ip = ctx.Connection.RemoteIpAddress?.ToString();
    var result = await sender.Send(new LoginCommand(req.Username, req.Password, ip),
    ct);
    return result.ToHttpResult(AuthResponse.From);    // Result → 200 or ProblemDetails
}

```

Why the endpoint is so small: it only translates HTTP ↔ command and lets the handler do the work. The `ToHttpResult` helper maps a `Result` onto either `200 OK` or an RFC-7807 `ProblemDetails` with the right status code, so HTTP concerns stay out of the business logic.

The protected test endpoint

src/Api/Endpoints/MemberEndpoints.cs (essence)

```

var group = app.MapGroup("/api/v1/members").RequireAuthorization();    // valid JWT
required
group.MapGet("/me", (ClaimsPrincipal user) => Results.Ok(new MeResponse(
    user.FindFirstValue(JwtRegisteredClaimNames.Sub),
    user.FindFirstValue(JwtRegisteredClaimNames.UniqueName),
    /* email, name */,
    user.FindAll("lob").Select(c => c.Value).ToArray(),
    user.FindAll("plan").Select(c => int.Parse(c.Value)).ToArray())));

```

Why include this? It is the simplest possible "downstream service": it requires a valid token and reads the `lob` / `plan` claims back out — exactly how a real microservice would authorize a member. It's your end-to-end proof that a token works.

6.3 Validating incoming tokens

src/Api/Infrastructure/ConfigureJwtBearerOptions.cs (essence)

```

options.TokenValidationParameters = new TokenValidationParameters
{
    ValidateIssuer = true,          ValidIssuer = _jwt.Issuer,
    ValidateAudience = true,       ValidAudience = _jwt.Audience,
    ValidateIssuerSigningKey = true, IssuerSigningKey = keyProvider.PublicKey,    //
    public key only
    ValidateLifetime = true,        ClockSkew = TimeSpan.FromMinutes(1),
    ValidAlgorithms = [SecurityAlgorithms.RsaSha256],    // pin RS256 – reject anything
    else
    RequireSignedTokens = true,     RequireExpirationTime = true,
};
options.MapInboundClaims = false;    // keep sub/lob/plan as-is

```

Why pin the algorithm? A famous JWT attack swaps the algorithm to `none` or downgrades RS256 to HS256 (using the public key as an HMAC secret). Explicitly allowing only `RS256` and requiring signed tokens shuts both attacks down.

6.4 Cross-cutting protections

Concern	Where	What it does
Rate limiting	<code>DependencyInjection.AddRateLimiting</code>	Per-IP fixed window (10/min) on auth endpoints to slow brute force / credential stuffing.
Security headers	<code>SecurityHeadersMiddleware</code>	nosniff, X-Frame-Options DENY, no-referrer, permissions-policy; removes Server header.
Global errors	<code>GlobalExceptionHandler</code> (<code>ExceptionHandler</code>)	Validation → 400 with field errors; anything else → generic 500. No stack traces leak.
CORS	<code>AddCors</code>	Only configured origins, only GET/POST — the Member Portal, not the whole internet.

7. Security design deep-dive

Threat	Mitigation in this codebase
Stolen signing key forges tokens everywhere	Asymmetric RS256 ; only this API holds the private key, others verify with the public key (JWKS).
Token replay / long-lived access	Access tokens expire in ~15 min; refresh tokens rotate on every use.
Stolen refresh token	Stored only as a SHA-256 hash; reuse of a rotated token revokes the entire chain (theft detection).
Password database leak	Only salted hashes are stored; verification is constant-time.
Brute force / credential stuffing	Per-IP rate limiting + account lockout after N failed attempts.
Username discovery (enumeration)	Identical generic 401 for all failures + dummy-hash to equalise timing.
JWT "alg" downgrade / none attack	Algorithm pinned to RS256; signed tokens required.
Misconfiguration reaching production	Options validated and signing key built at startup (fail-fast).
Information leakage via errors	RFC-7807 ProblemDetails; no internal details or stack traces returned.
Transport & browser attacks	HTTPS redirection, HSTS (non-dev), strict security headers, locked-down CORS.

The JWT claims issued

`sub` (member id), `jti` (token id), `unique_name`, `email`, `given_name`, `family_name`, standard `iss/aud/iat/nbf/exp`, and one `lob` claim per line of business plus one `plan` claim per plan id. Downstream services read `lob / plan` to authorize what a member may see.

8. Configuration, mocking & switching to the real DB

All knobs live in `appsettings.json` (overridable per environment and via user-secrets / environment variables / Key Vault).

Section	Controls
<code>Database:Provider</code>	<code>InMemory</code> (mock, default) or <code>SqlServer</code> (real DB).
<code>Database:SeedMockData</code>	Whether to seed demo members (true in Dev).
<code>ConnectionStrings:AuthDb</code>	The real member database connection (SqlServer mode only).
<code>Jwt</code>	Issuer, Audience, token lifetimes, and the RSA key source.
<code>PasswordHashing</code>	Algorithm / salt placement / encoding / iterations to match the legacy DB.
<code>AccountLockout</code>	Max failed attempts & lockout duration.
<code>Cors:AllowedOrigins</code>	Which front-ends may call the API.

Going to the real database — the three steps

1. Set `Database:Provider = SqlServer`, `SeedMockData = false`, and a real `ConnectionStrings:AuthDb`.
2. Adjust the EF mappings in `Infrastructure/Persistence/Configurations/*` to the real table/column names.
3. Set `PasswordHashing` to match exactly how the existing system hashes passwords.

Moving to .NET 10

The target framework is centralised in `Directory.Build.props`. Change `net8.0` → `net10.0`, point `global.json` at a .NET 10 SDK, and remove the `RollForward` hint in the API project. One line per concern.

9. Project-setup best practices

File	Purpose & why
global.json	Pins the .NET 8 SDK so every machine/CI builds with the same toolchain — no "works on my machine".
Directory.Build.props	One place for shared MSBuild settings (TargetFramework, Nullable, ImplicitUsings, analysis level). Keeps the four .csproj files tiny and consistent; makes the .NET 10 move a one-line change.
Directory.Packages.props	Central Package Management. Every NuGet version is declared once; projects reference packages without versions. Prevents version drift and makes upgrades trivial.
.editorconfig	Enforces consistent formatting & naming across the team, checked at build time.

These four are part of the "set up a new .NET project the right way" practices. We applied `global.json`, `Directory.Build.props`, Central Package Management and `.editorconfig`; stricter analyzers, container orchestration and a CI pipeline were intentionally left as opt-in.

10. How to run & test

```
# Build & run (uses the per-user .NET 8 SDK pinned by global.json)
& "$env:USERPROFILE\.dotnet\dotnet.exe" run --project "src\Api\AuthApi.Api.csproj"
# Swagger UI: http://localhost:5155/swagger
```

Demo members (Development)

Username	Password	LOBs	Plan IDs
jdoe	P@ssw0rd!	DENTAL, VISION	1001, 2001
asmith	Secret123!	MEDICAL	3001

```
$base = "http://localhost:5155"
$login = Invoke-RestMethod "$base/api/v1/auth/login" -Method Post -ContentType
"application/json" `
    -Body '{"username":"jdoe","password":"P@ssw0rd!"}'
Invoke-RestMethod "$base/api/v1/members/me" -Headers @{ Authorization = "Bearer
$($login.accessToken)" }
```

Try the security behaviour: wrong password → 401; `/me` with no token → 401; reuse a rotated refresh token → 401 (chain revoked); paste a token into jwt.io to see the `lob` / `plan` claims and the `kid`.

11. Glossary

Term	Meaning
JWT	JSON Web Token — a signed, self-describing token carrying claims about the user.
Access token	Short-lived JWT proving identity to APIs.
Refresh token	Longer-lived opaque token used to get new access tokens without re-login.
Claim	A key/value statement inside a token (e.g. <code>sub</code> , <code>lob</code>).
RS256	RSA signature with SHA-256 — asymmetric (private key signs, public key verifies).
JWKS	JSON Web Key Set — the public keys, published so others can validate tokens.
CQRS	Command/Query Responsibility Segregation — model each operation as an explicit request.
MediatR	Library that dispatches a command to its single handler (and runs pipeline behaviors).
LOB	Line of Business (e.g. DENTAL, VISION) a member is associated with.
Salt	Random per-user value mixed into a password before hashing, so identical passwords hash differently.